

Progetto di Intelligent Robot Navigation

Relazione Tecnica

Gruppo 01

Federica de Maio Noemi Biancamano Alex Cuciniello Davide D'Acunto

Sommario

In questo documento viene presentata la relazione tecnica del progetto di Intelligent Robot Navigation del gruppo 01. L'obiettivo posto è lo sviluppo di un sistema di navigazione intelligente per TurtleBot4, il quale permetta a questo di muoversi autonomamente in un ambiente conosciuto, alla ricerca di un *marker* specifico. Da lì, il robot deve essere in grado di seguirlo, evitando ostacoli e operando anche in presenza di brevi occlusioni. Il tutto dev'essere realizzato nel minor tempo possibile all'interno di un percorso predefinito, con l'obiettivo di minimizzare la funzione temporale come specificata di seguito:

$$T_{\text{tot}} = \alpha \cdot t_{\text{espl}} + \beta \cdot t_{\text{goal}} + (N_{\text{penalty}} \cdot t_{\text{penalty}}), \\ \alpha < \beta, N_{\text{penalty}} \geq 0$$

Indice

1	Introduzione	2
2	Architettura	3
2.1	Aruco_Reader	3
2.2	Esplorazione	3
2.3	Inseguimento	3
2.3.1	Path Planner	4
2.3.2	Path Smoother	4
2.3.3	Path Controller	4
2.3.4	Compute_Pose	4
2.3.5	Predictive_Follower_FSM	5

1 Introduzione

L'inseguimento di un *marker* da parte di un robot è una problematica comune in ambito di robotica mobile, molto presente in letteratura e con un ampio ventaglio di applicazioni pratiche. In questo progetto alcune idee sono state prese da lavori precedenti, tra le quali citiamo:

- **Aruco & OpenCV:** Come *marker* si è scelto di adoperare un ArUcoo, tipicamente utilizzato in applicazioni di *Computer Vision*, difatti anche supportato da OpenCV, una libreria open-source per la visione artificiale.
- **Nav2:** In supporto alla navigazione del robot, Nav2 permette di automatizzare il processo di pianificazione del percorso, ricalcolo in caso di ostacoli imprevisti, e di esecuzione del movimento, con un'architettura modulare e molto più efficiente rispetto a soluzioni personalizzate.

Queste sono state le scelte principali che hanno spinto lo sviluppo del progetto, verranno quindi descritte in dettaglio le loro integrazioni all'interno del progetto, insieme alle componenti sviluppate ad-hoc per il raggiungimento dell'obiettivo.

Il progetto, e quindi anche il documento, suddivide il problema in due aree principali: **Eplorazione** e **Inseguimento**. Nella prima, il robot deve esplorare l'ambiente alla ricerca del *marker*, mentre nella seconda, una volta individuato, deve seguirlo fino al raggiungimento dell'area obiettivo, evitando ostacoli e operando anche in presenza di brevi occlusioni.

2 Architettura

Come accennato precedentemente, il progetto è suddiviso in due aree principali, entrambe tuttavia dipendenti da un'architettura software comune e da una componente condivisa: il nodo `Aruco_Reader`. Esso è responsabile di osservare l'area dai frame della telecamera, identificare la presenza dei *marker* e focalizzarsi su quello d'interesse, restituendone la posizione relativa. Inoltre, entrambi i processi condividono la mappa dell'ambiente, che viene utilizzata per la pianificazione del percorso e per la *obstacle avoidance*.

2.1 Aruco_Reader

Il nodo `Aruco_Reader` è stato sviluppato mediante la libreria OpenCV, che fornisce strumenti per la rilevazione e il riconoscimento dei *marker* ArUco. Si comporta come *subscriber* sul *topic* della telecamera, elaborando i frame in tempo reale per identificare la presenza dei *marker*. Una volta individuato, ne pubblica la posizione relativa rispetto al robot su un *topic* dedicato, che viene poi utilizzato sia durante l'esplorazione che durante l'inseguimento per guidare il movimento del robot.

Questa centralizzazione della funzionalità di rilevamento è stata pensata per semplificare l'architettura e uniformare le informazioni tra i due processi principali.

2.2 Esplorazione

2.3 Inseguimento



Figura 1: Diagramma a blocchi dell'area d'inseguimento, con le interazioni tra i nodi principali.

Nell'area d'inseguimento, il robot utilizza le informazioni fornite dal nodo `Aruco_Reader` per pianificare un percorso verso il *marker* identificato. Nav2 viene utilizzato per gestire la pianificazione del percorso e l'esecuzione del movimento, rilassando il problema dell'elusione degli ostacoli.

Il robot deve essere in grado di seguire il *marker* anche in presenza di brevi occlusioni, per cui è stato implementato un meccanismo di predizione basato su filtro di Kalman e la velocità del *marker* prima dell'occlusione, permettendo al robot di continuare a muoversi nella direzione prevista fino a quando il *marker* non viene nuovamente rilevato.

Inoltre, si desidera implementare una strategia di *replanning* dinamico, che consenta al robot di adattare il percorso in tempo reale in risposta a ostacoli imprevisti, anche durante l'inseguimento, tuttavia senza interrompere il movimento verso il *marker*. Il processo di pianificazione del percorso è estremamente lento, e in caso di assenza di percorso impone al robot di fermarsi e ricalcolare il nuovo obiettivo per alcuni secondi, con conseguente aumento

del tempo totale e quindi della penalità. Invece, il percorso viene calcolato in background, e una volta pronto, viene semplicemente sostituito a quello attuale, senza interrompere il continuo flusso di movimento verso il *marker*.

2.3.1 Path Planner

La pianificazione del percorso avviene mediante un *action server* di Nav2 dedicato denominato `Compute_Path_To_Pose`, che riceve come input la posa stimata del *marker* e restituisce un percorso pianificato verso essa a partire da quella attuale del robot. In particolare, come pianificatore del percorso viene utilizzato il plugin `Smac2D`, adoperato per la sua efficienza e capacità di gestire ambienti dinamici, e tenersi alla larga dagli ostacoli statici e dai bordi.

2.3.2 Path Smoother

Il processo che rende più fluido il percorso pianificato avviene mediante un altro *action server* di Nav2 dedicato denominato `Smooth_Path`, che riceve come input il percorso pianificato e restituisce un percorso più fluido, con meno punti e curve più dolci. In particolare, come algoritmo di smoothing viene utilizzato `Savitzky-Golay`, adoperato in quanto estremamente efficiente, veloce e efficace.

2.3.3 Path Controller

Il processo di esecuzione del percorso avviene mediante un altro *action server* di Nav2 dedicato denominato `Follow_Path`, che riceve come input il percorso fluido e lo esegue. In particolare, come controller del percorso viene utilizzato `RegulatedPurePursuit`, adoperato per la sua capacità di gestire curve strette e di tenersi sempre rivolto verso il *marker*, anche in presenza di curve strette o di ostacoli imprevisti. Ciò risulta fondamentale per non perdere di vista il *marker* durante l'inseguimento, e per evitare di dover ricalcolare il percorso in caso di ostacoli imprevisti, con conseguente aumento del tempo totale e quindi della penalità.

2.3.4 Compute_Pose

Una volta che il nodo `Aruco_Reader` pubblica una posa, diventa compito di `Compute_Pose` la trasposizione di questa all'interno della mappa, con relativa applicazione del filtro di *Kalman*.

Il nodo si sottoscrive al *topic* `/aruco/pose`, nella quale viene pubblicata la posa relativa al robot del *marker* *AruCo*, nel sistema di riferimento adoperato da OpenCV. Ottenuta la posa, essa viene manipolata per essere trasposta all'interno della mappa, con le dovute rotazioni attorno agli assi e scostamento rispetto alla posizione del robot nella mappa. Successivamente, un'altra operazione determina la posizione frontale nella quale si desidera posizionare il robot. Infine, una rotazione di 180° attorno all'asse perpendicolare alla mappa rivolge il robot nella direzione del *marker*.

A causa dell'errore di misurazione della telecamera non è possibile affidarsi esclusivamente a essa per determinare la posa da raggiungere. Piuttosto, la posa "frontale" viene passata ad

un filtro di *Kalman* che, grazie ai delta temporali e alla conoscenza pregressa, effettua una stima più dolce che pubblica su `/estimated/pose`. Ciò risulta estremamente utile quando il robot è in movimento o sta ruotando, siccome le misurazioni diventano molto più rumorose a causa del tremolio della telecamera.

2.3.5 Predictive_Follower_FSM

L'inseguimento viene governato dal seguente nodo, mediante una macchina a stati finiti che rappresenta le diverse fasi dell'inseguimento. Le transizioni avvengono sulla base di due eventi:

- **/estimated/pose:** L'arrivo di una nuova posa stimata è un evento in grado di provocare una transizione di stato, in particolare quando il robot è in stato di `IDLE` oppure `FOLLOWING_PATH`. Al contrario, l'assenza delle pose stimate (misurata mediante il numero di pose vuote) può provocare una transizione di stato verso `LOST`, che rappresenta la condizione in cui il robot non è più in grado di seguire il *marker*.
- **Action callback:** Ciascuna action che chiama interfacce di Nav2 genera inevitabilmente una risposta, che può essere di successo o di fallimento. Entrambi i casi sono eventi in grado di provocare una transizione di stato, in avanti se l'azione è stata completata con successo, indietro se invece è fallita. Eccetto casi particolari di concorrenza tra percorsi o laddove subentri lo stato `LOST`, esse rappresentano l'evoluzione naturale della FSM.

IDLE In questo stato il robot non sta seguendo alcun percorso, e attende l'arrivo di una nuova posa stimata dal nodo `Compute_Pose`. Una volta ricevuta, il robot comincia il processo di pianificazione del percorso verso la posa stimata. Il primo *action server* che viene interpellato è quello di `Compute_Path_To_Pose` che calcola un percorso verso la posa stimata, dopo l'invio della richiesta lo stato passa a `COMPUTING_PATH`.

COMPUTING_PATH Nel suddetto stato il robot attende che riceva informazioni sulla gestione del *goal* da parte del `Compute_Path_To_Pose action server`. Se esso si prende carico del *goal*, il robot passa allo stato `SMOOTHING_PATH` e associa la *callback* di gestione della risposta con una chiamata all'altro *action server* `Smooth_Path`, altrimenti ritorna in `IDLE` e attende una nuova posa stimata.

SMOOTH_PATH Analogamente allo stato precedente, la transizione avviene in base alla risposta dell'*action server* `Smooth_Path` sul *goal*, andando o indietro all'`IDLE` o in avanti a `FOLLOWING_PATH`. Tuttavia, come prima questa transizione avviene "in anticipo": il robot non attende la risposta dell'*action server* per passare allo stato successivo, ma lo fa immediatamente dopo l'accettazione della richiesta mediante risposta sul *goal*, associando la *callback* di gestione della risposta con una chiamata all'*action server* `Follow_Path`. Ciò rende la FSM più simile da una macchina di Moore che di Mealy, lo storico degli ingressi sono di per sé la codifica dello stato. Ad ogni modo, la risposta dell'*action server* `Smooth_Path` è lo stesso percorso reso più fluido grazie al *path smoother*, che viene infine passato come percorso da seguire.

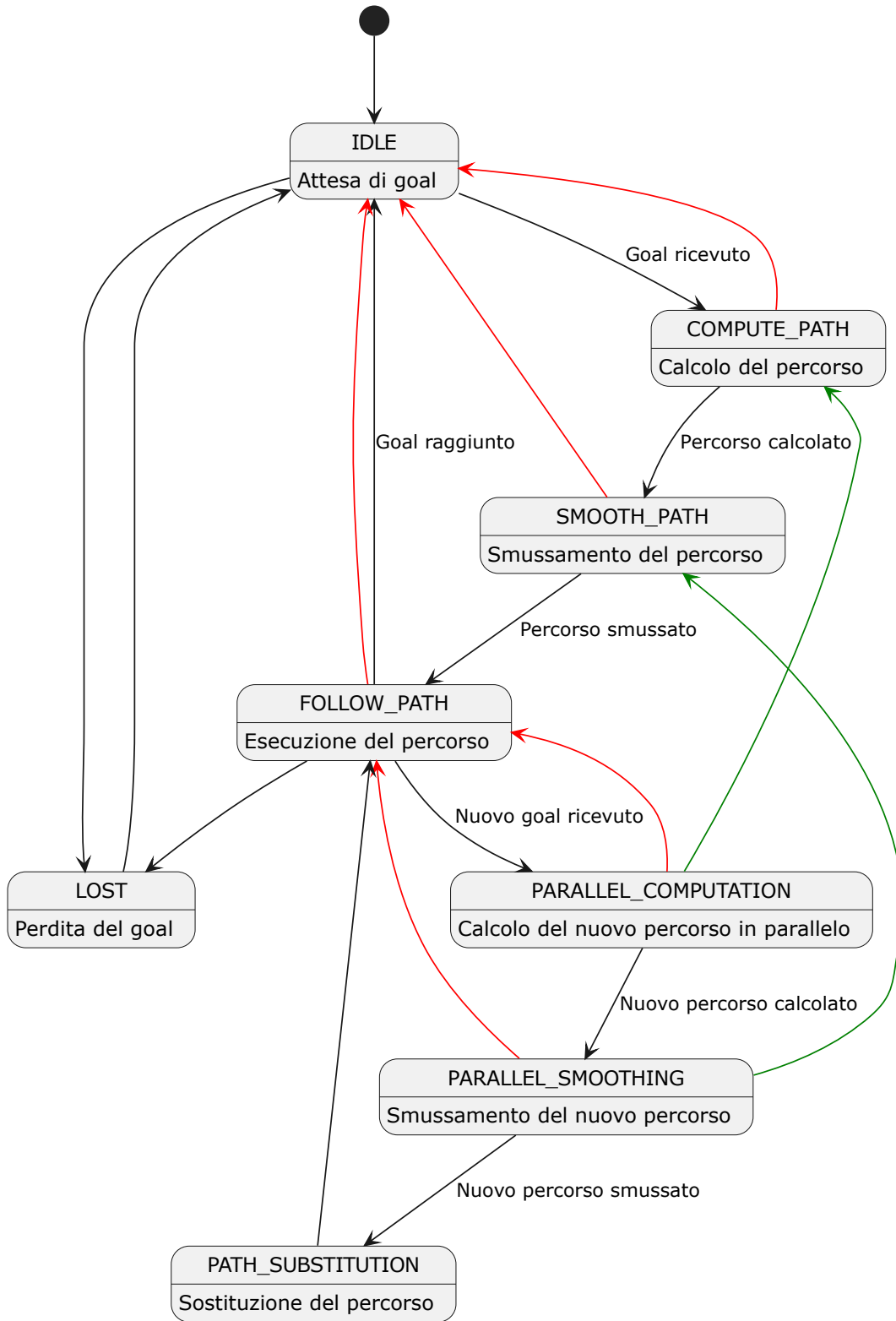


Figura 2: Diagramma a stati finiti del nodo `Predictive_Follower_FSM`, le frecce rosse indicano errori all'interno del nodo, quelle verdi la fine di un percorso precedentemente pianificato.

FOLLOWING_PATH Ricevuto il percorso da seguire, il robot interpella Nav2 attraverso l'*action server Follow_Path*, e in questo caso le transizioni di stato evolvono in base alla tipologia di *callback* ricevuta.

Se la *callback* è sul goal, non vi sono transizioni di stato, a meno che non sia di errore, in tal caso il robot ritorna in **IDLE** e attende una nuova posa stimata. In caso di successo, alla *callback* di risposta viene associato un ventaglio di possibili transizioni di stato:

- **IDLE**: Il robot ha raggiunto il *goal* e ritorna in attesa di una nuova posa stimata.
- **PARALLEL_COMPUTATION**: Il robot ha ricevuto una nuova posa stimata, e quindi comincia il processo di pianificazione del percorso verso la nuova posa stimata. In questo caso, il robot non interrompe il movimento verso il *goal* attuale, ma continua a seguirlo fino a quando non termina l'azione, oppure fino a quando non riceve un nuovo percorso.
- **PARALLEL_* → ***: Il robot era in uno stato di pianificazione del percorso successivo, tuttavia il percorso attuale è stato completato, e quindi il robot può evitare di effettuare la sostituzione del percorso, e prepararsi direttamente a seguire il nuovo percorso.

PARALLEL_COMPUTATION & PARALLEL_SMOOTHING Due stati analoghi a **COMPUTING_PATH** e **SMOOTHING_PATH**, ma con la differenza che il robot non interrompe il movimento verso il *goal* attuale, ma continua a seguirlo fino a quando non termina l'azione, oppure fino a quando non riceve un nuovo percorso. In caso di errore nella pianificazione del percorso, il robot ritorna in **FOLLOWING_PATH** e continua a seguire il percorso attuale. In caso di successo della pipeline, il robot passa in **SUBSTITUTE_PATH** e sostituisce il percorso attuale con quello nuovo.

SUBSTITUTE_PATH In questo stato il robot sostituisce il percorso attuale con quello nuovo, e ritorna in **FOLLOWING_PATH** per continuare a seguire il nuovo percorso. Questo stato intermedio è necessario per discriminare tra il caso in cui Nav2 rigetti il percorso passato come nuovo, e il caso in cui lo accetti: sebbene in entrambi i casi la transizione di stato sia verso **FOLLOWING_PATH**, nel primo caso il robot continuerà a seguire il percorso attuale, mentre nel secondo caso seguirà il nuovo percorso. Ciò tuttavia non cambia il comportamento del robot, né della FSM, ma è utile per la gestione dei log, per il debug e per eventuali future estensioni della FSM che permettano di gestire in maniera differente i due casi.

LOST La transizione verso questo stato avviene quando il robot non riceve più pose stimate dal nodo **Compute_Pose**, a prescindere dallo stato in cui si trova. In questo caso, il robot interrompe il movimento verso il *goal* attuale, e comincia a ruotare su sé stesso alla ricerca del *marker*. Una volta che il robot riceve una nuova posa stimata, ritorna in **IDLE** e comincia nuovamente il processo di pianificazione del percorso verso la nuova posa stimata.